

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____

Преподаватель _____ С.В. Умбетов

«___» _____ 2026 г.

Отчет по лабораторной работе №2
по дисциплине «Инженерно-техническая поддержка
сопровождения интеллектуальных информационных систем»
«Основы работы с GIT»
ЛР 09.02.08.02.002

Студент группы	<u>1ИИС-41</u>	<u>Н.М. Аникин</u>
	группа	и.о., фамилия
Преподаватель	<u>старший преподаватель, доцент</u>	<u>С.В. Умбетов</u>
	должность, ученая степень	и.о., фамилия

Барнаул 2026

Лабораторная работа №2

Основы работы с GIT

Цели и задачи работы: освоить базовые и продвинутые приёмы работы с распределённой системой контроля версий Git в операционной системе Linux. Сформировать практические навыки организации версионного контроля при разработке программного обеспечения и понимание роли Git в командной работе.

- изучить процесс установки и настройки Git;
- освоить основные команды для работы с локальным репозиторием;
- научиться управлять состояниями файлов, работать с ветками, выполнять слияния и разрешать конфликты.

Задание к работе: выполнить задания 1–20 в соответствии с условиями. Ответить на контрольные вопросы.

Задание принял: _____



Аникин Н.М.

Подпись

Ход работы

Задание 1. Откройте терминал и проверьте наличие Git в системе командой `git --version`.

```
anikin_1iis41@ubuntu26041:~$ git --version
git version 2.53.0
```

Рисунок 1 – проверка наличия Git в системе

В системе уже установлен по умолчанию Git версии 2.53.0

Задание 2. Определите свой дистрибутив Linux командой `cat /etc/os-release`.

```
anikin_1iis41@ubuntu26041:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04.1 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04.1 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
anikin_1iis41@ubuntu26041:~$
```

Рисунок 2 – определение моего дистрибутива Linux

Задание 3. Установите Git с помощью менеджера пакетов вашего дистрибутива (`sudo apt-get install git` для Ubuntu/Debian, `sudo dnf install git` для Fedora, `sudo yum install git` для CentOS/RHEL, `sudo pacman -S git` для Arch Linux).

```

anikin_1iis41@ubuntu26041:~$ sudo apt-get update
Hit:1 http://ru.archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://ru.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://ru.archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:5 http://ru.archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [716 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [550 kB]
Get:7 http://ru.archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [167 kB]
Get:8 http://ru.archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [97.9 kB]
Get:9 http://ru.archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [486 kB]
Get:10 http://ru.archive.ubuntu.com/ubuntu resolute-updates/restricted Translation-en [97.7 kB]
Get:11 http://ru.archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [315 kB]
Get:12 http://ru.archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [101 kB]
Get:13 http://ru.archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [201 kB]
Get:14 http://ru.archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 Packages [11.9 kB]
Get:15 http://ru.archive.ubuntu.com/ubuntu resolute-updates/multiverse Translation-en [3,276 B]
Get:16 http://ru.archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Components [1,060 B]
Get:17 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [129 kB]
Get:18 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.7 kB]
Get:19 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [445 kB]
Get:20 http://security.ubuntu.com/ubuntu resolute-security/restricted Translation-en [89.7 kB]
Get:21 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [191 kB]
Get:22 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [60.4 kB]
Get:23 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [55.5 kB]
Fetched 4,178 kB in 2s (2,727 kB/s)
Reading package lists... Done
anikin_1iis41@ubuntu26041:~$ sudo apt-get install git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.53.0-1ubuntu1).
Solving dependencies... Done
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-30 linux-headers-7.0.0-30-generic linux-image-unsigned-7.0.0-30-generic linux-main-modules-zfs-7.0.0-30
  linux-modules-7.0.0-30-generic linux-tools-7.0.0-30 linux-tools-7.0.0-30-generic
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 49 not upgraded.

```

Рисунок 3 – установка Git с помощью менеджера пакетов

Задание 4. Повторно проверьте версию Git командой `git --version` для подтверждения успешной установки.

```

anikin_1iis41@ubuntu26041:~$ git --version
git version 2.53.0

```

Рисунок 4 – повторная проверка версии Git

В результате установки не применено никаких изменений, что говорит о том, что в системе уже была установлена наиболее актуальная версия Git.

Задание 5. Настройте имя пользователя командой `git config --global user.name "Ваше Имя"`.

```

anikin_1iis41@ubuntu26041:~$ git config --global user.name "anikin_1iis41"

```

Рисунок 5 – настройка имени пользователя

Задание 6. Настройте адрес электронной почты командой `git config --global user.email your.email@example.com`.

```

anikin_1iis41@ubuntu26041:~$ git config --global user.email your.email@example.com

```

Рисунок 6 – настройка адреса электронной почты

Задание 7. Проверьте применённые настройки командой `git config --list`.

```
anikin_1iis41@ubuntu26041:~$ git config --list
user.name=anikin_1iis41
user.email=your.email@example.com
core.repositoryformatversion=0
core.filemode=true
core.bare=false
core.logallrefupdates=true
anikin_1iis41@ubuntu26041:~$
```

Рисунок 7 – проверка применённых настроек

Задание 8. Создайте рабочую директорию командой `mkdir git-practice` и перейдите в неё командой `cd git-practice`.

```
anikin_1iis41@ubuntu26041:~$ mkdir git-practice
anikin_1iis41@ubuntu26041:~$ cd git-practice
anikin_1iis41@ubuntu26041:~/git-practice$
```

Рисунок 8 – создание рабочей директории и переход в неё

Задание 9. Инициализируйте локальный репозиторий командой `git init`.

```
anikin_1iis41@ubuntu26041:~/git-practice$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to 'main' in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:   git config --global init.defaultBranch <name>
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:   git branch -m <name>
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/anikin_1iis41/git-practice/.git/
anikin_1iis41@ubuntu26041:~/git-practice$ _
```

Рисунок 9 – инициализация локального репозитория

Задание 10. Убедитесь в создании скрытой папки `.git` командой `ls -la`.

```
anikin_1iis41@ubuntu26041:~/git-practice$ ls -la
total 12
drwxrwxr-x 3 anikin_1iis41 anikin_1iis41 4096 Oct  4 12:46 .
drwxr-x--- 8 anikin_1iis41 anikin_1iis41 4096 Oct  4 12:45 ..
drwxrwxr-x 6 anikin_1iis41 anikin_1iis41 4096 Oct  4 12:46 .git
anikin_1iis41@ubuntu26041:~/git-practice$ _
```

Рисунок 10 – проверка создания скрытой папки `.git`

Задание 11. Создайте несколько файлов, например `readme.txt`, `script.sh` и `data.json`, с помощью команды `touch`.

```
anikin_1iis41@ubuntu26041:~/git-practice$ touch readme.txt script.sh data.json
```

Рисунок 11 – создание файлов

Задание 12. Проверьте статус репозитория командой `git status` и определите состояние созданных файлов (untracked).

```
anikin_1iis41@ubuntu26041:~/git-practice$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        data.json
        readme.txt
        script.sh

nothing added to commit but untracked files present (use "git add" to track)
anikin_1iis41@ubuntu26041:~/git-practice$ _
```

Рисунок 12 – проверка статуса репозитория

Файлы созданы, но не добавлены в репозиторий, поэтому они находятся в состоянии untracked.

Задание 13. Добавьте в состояние staged только текстовые файлы командой `git add *.txt`.

```
anikin_1iis41@ubuntu26041:~/git-practice$ git add *.txt
anikin_1iis41@ubuntu26041:~/git-practice$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   readme.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        data.json
        script.sh

anikin_1iis41@ubuntu26041:~/git-practice$ _
```

Рисунок 13 – добавление в состояние staged текстового файла и проверка статуса

Задание 14. Добавьте все остальные файлы командой `git add -A` и повторно проверьте статус командой `git status`.

```

anikin_1iis41@ubuntu26041:~/git-practice$ git add -A
anikin_1iis41@ubuntu26041:~/git-practice$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   data.json
        new file:   readme.txt
        new file:   script.sh

anikin_1iis41@ubuntu26041:~/git-practice$ _

```

Рисунок 14 – добавление всех остальных файлов и проверка статуса

Задание 15. Создайте первый коммит командой `git commit -m "initial commit: added base files"`.

```

anikin_1iis41@ubuntu26041:~/git-practice$ git commit -m "initial commit: added base files"
[master (root-commit) 82c1406] initial commit: added base files
 3 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 data.json
 create mode 100644 readme.txt
 create mode 100644 script.sh
anikin_1iis41@ubuntu26041:~/git-practice$ _

```

Рисунок 15 – создание первого коммита

Задание 16. Просмотрите историю коммитов командой `git log` и зафиксируйте метаданные коммита.

```

anikin_1iis41@ubuntu26041:~/git-practice$ git log
commit 82c140666737315c2896b2d855ea62009ab7f0a5 (HEAD -> master)
Author: anikin_1iis41 <your.email@example.com>
Date:   Sun Oct 4 13:03:34 2026 +0000

    initial commit: added base files
anikin_1iis41@ubuntu26041:~/git-practice$ _

```

Рисунок 16 – просмотр истории коммитов

В истории зафиксированы следующие метаданные коммита:

- 1) 82c140666737315c2896b2d855ea62009ab7f0a5 – уникальный идентификатор коммита;
- 2) HEAD -> master – указатель текущей ветки;
- 3) Author: anikin_1iis41 <your.email@example.com> - имя и почта создателя коммита;
- 4) Date: Sun Oct 4 13:03:34 2026 +0000 – время создания коммита;
- 5) initial commit: added base files – комментарий от автора.

Задание 17. Внесите изменения в файл `readme.txt` и просмотрите различия командой `git diff`.

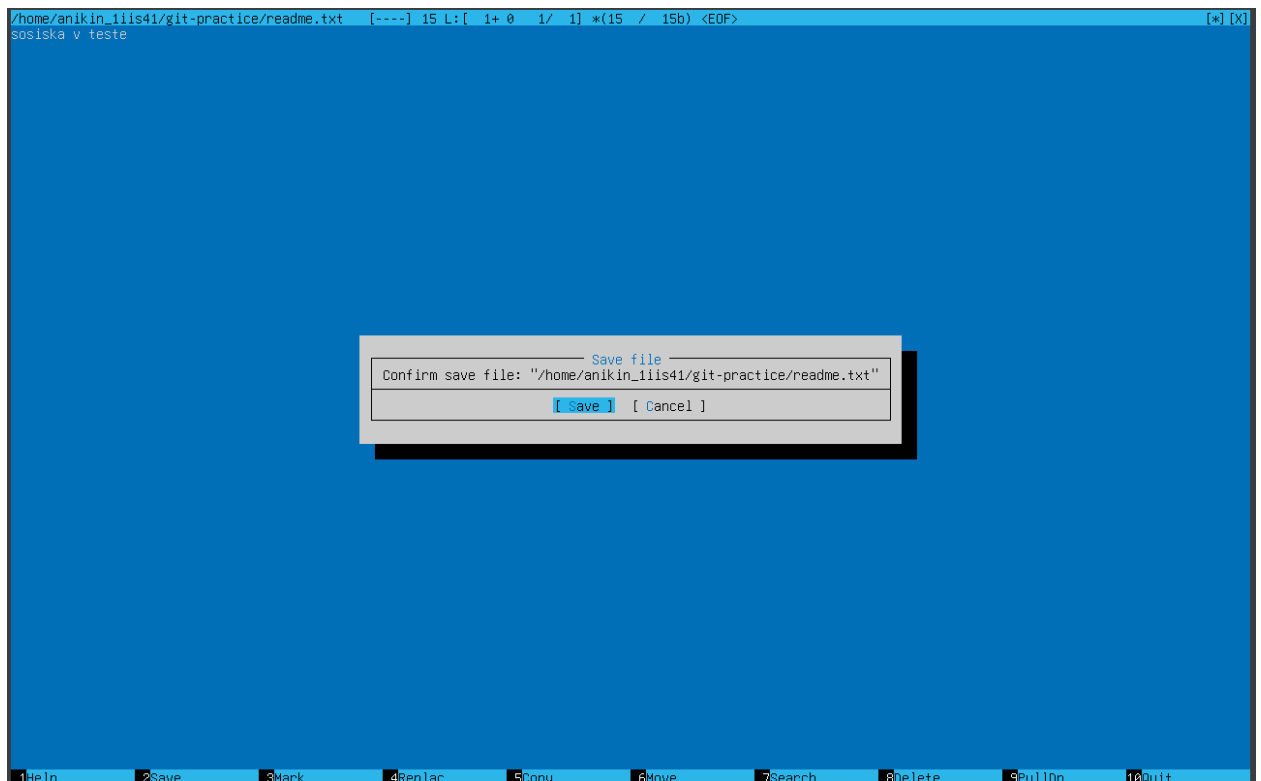


Рисунок 17 – изменения файла `readme.txt` с помощью редактора `mc`

```
anikin_1iis41@ubuntu26041:~/git-practice$ git diff
diff --git a/readme.txt b/readme.txt
index e69de29..3469830 100644
--- a/readme.txt
+++ b/readme.txt
@@ -0,0 +1 @@
+sosiska v teste
\ No newline at end of file
anikin_1iis41@ubuntu26041:~/git-practice$ _
```

Рисунок 18 – просмотр различий

Задание 18. Создайте файл `.gitignore` командой `touch .gitignore` и внесите в него шаблоны `*.log`, `temp/`, `*.tmp`, `.cache/`.

```
anikin_1iis41@ubuntu26041:~/git-practice$ touch .gitignore
```

Рисунок 19 – создание файла `.gitignore`

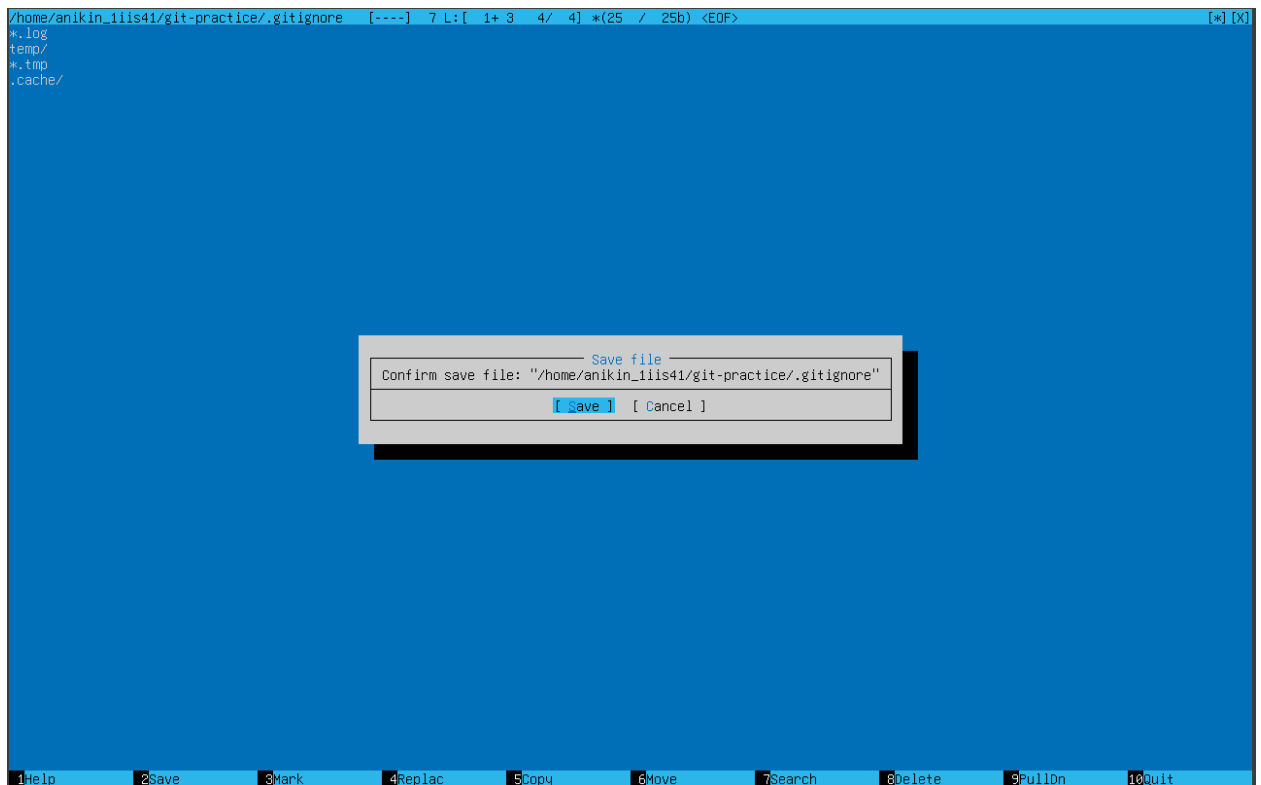


Рисунок 20 – внесение шаблонов в .gitignore

Задание 19. Создайте новую ветку командой `git checkout -b feature/update-readme` и проверьте список веток командой `git branch -a`.

```
anikin_1iis41@ubuntu26041:~/git-practice$ git checkout -b feature/update-readme
Switched to a new branch 'feature/update-readme'
anikin_1iis41@ubuntu26041:~/git-practice$ git branch -a
* feature/update-readme
  master
anikin_1iis41@ubuntu26041:~/git-practice$ _
```

Рисунок 21 – создание новой ветки и проверка списка веток

Задание 20. Внесите изменения в файл `readme.txt` в новой ветке, закоммитьте их, переключитесь на `master` командой `git checkout master`, выполните слияние командой `git merge feature/update-readme` и проанализируйте итоговую историю командой `git log --oneline --graph --all`.

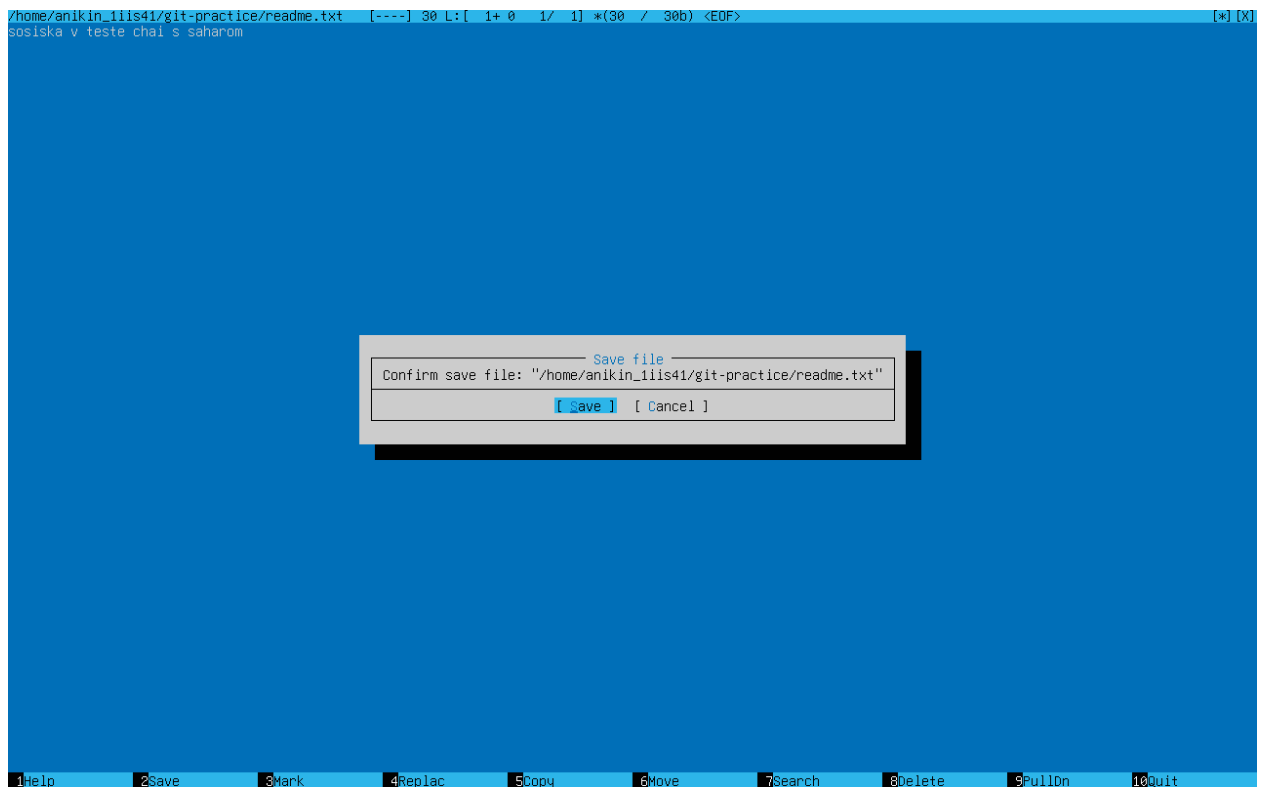


Рисунок 22 – изменение файла readme.txt в новой ветке

```
anikin_1iis41@ubuntu26041:~/git-practice$ git commit -a -m "second commit: edited readme.txt"
[feature/update-readme fd4ec39] second commit: edited readme.txt
1 file changed, 1 insertion(+)
anikin_1iis41@ubuntu26041:~/git-practice$
```

Рисунок 23 – коммит изменений

```
anikin_1iis41@ubuntu26041:~/git-practice$ git checkout master
Switched to branch 'master'
anikin_1iis41@ubuntu26041:~/git-practice$ git merge feature/update-readme
Updating 82c1406..fd4ec39
Fast-forward
 readme.txt | 1 +
1 file changed, 1 insertion(+)
```

Рисунок 24 – переключение на мастер-ветку и выполнение слияния с новой веткой

```
anikin_1iis41@ubuntu26041:~/git-practice$ git log --oneline --graph --all
* fd4ec39 (HEAD -> master, feature/update-readme) second commit: edited readme.txt
* 82c1406 initial commit: added base files
anikin_1iis41@ubuntu26041:~/git-practice$
```

Рисунок 25 – итоговая история

Контрольные вопросы

1. Что такое Git и для чего он используется?

Git — это распределённая система контроля версий, предназначенная для управления исходным кодом. Она используется для хранения кода в едином месте, удобного объединения изменений от разных разработчиков, хранения истории изменений с описанием и авторством, отката неудачных изменений и ревью изменений.

2. В чём разница между локальным и удалённым репозиторием?

Локальный репозиторий хранит всю информацию локально в проекте, в папке `.git`. Удалённый репозиторий предполагает хранение информации на удалённом сервере: локально хранится только копия проекта, изменения которой можно отправить (`git push`) в удалённый репозиторий.

3. Какая команда проверяет наличие Git в системе и его версию?

Наличие Git в системе и его версию проверяет команда `git --version`. Если команда возвращает корректный ответ с указанием версии, например `git version 2.x.x`, установка не требуется. В противном случае будет выведено сообщение об отсутствии команды, что свидетельствует о необходимости установки Git.

4. Какие команды используются для установки Git в Ubuntu, Fedora и Arch Linux?

Для Ubuntu: `sudo apt-get update`, а затем `sudo apt-get install git`

Для Fedora: `sudo dnf install git`

Для Arch Linux: `sudo pacman -S git`

5. Для чего нужна настройка `user.name` и `user.email` в Git?

Эти настройки необходимы, поскольку при создании коммита Git использует данную информацию для заполнения поля `Author`.

6. В чём разница между командами `git config --global` и `git config` без флага?

Команда `git config --global` нужна для настройки имени пользователя и адреса электронной почты для всех проектов, а `git config` без флага нужна для изменения автора для конкретного проекта.

7. Что создаётся в директории при выполнении команды `git init`?

Команда `git init` создает локальный репозиторий. После её выполнения, в директории, где находится консоль, будет создана папка `.git`

8. Для чего предназначена папка `.git` и можно ли её удалять?

`.git` — это папка, хранящая всю информацию о `git`-репозитории. Удалять её не следует

9. Какие четыре состояния файлов существуют в `Git`?

1) Неотслеживаемое (`untracked`): файл, созданный и не добавленный в репозиторий.

2) Изменённое (`modified`): при внесении изменений в файлы, уже добавленные в `git`-репозиторий.

3) Подготовленное (`staged`): выбранные необходимые файлы из изменённых.

4) Закоммиченное (`committed`): из подготовленных файлов создаётся коммит, и они переходят в `git`-репозиторий.

10. В чём разница между состояниями `untracked` и `modified`?

Состояние `untracked` указывает, что созданный файл ещё не был добавлен в репозиторий, в то время как `modified` указывает на внесение изменений в файлах, добавленных в репозиторий.

11. Чем отличается команда `git add .` от `git add -A` и `git add` ?

`git add .` — добавить все файлы из текущей папки и вложенных;

`git add -A` — добавить все файлы из состояния в `staged` (по сути аналогичен `git add .`;

`git add` — добавить только конкретный файл.

12. Что такое коммит и какие метаданные он содержит?

Коммит — это основной объект в системе контроля версий. Он содержит все изменения, внесённые за время его создания. Коммиты связаны между собой как односвязный список. Он содержит следующие метаданные: уникальный идентификатор, по которому его можно найти; имя автора коммита; дата создания коммита; комментарий, описывающий внесённые изменения.

13. Для чего нужна команда `git status` и какую информацию она отображает?

Команда `git status` нужна для просмотра текущего статуса. Она отображает текущую рабочую ветку, а также файлы в рабочей директории и их состояния.

14. Что показывает команда `git diff` и в каких случаях она применяется?

Команда `git diff` показывает внесённые изменения в файлы. Она применяется, когда имеются файлы в состоянии `modified`.

15. Для чего используется файл `.gitignore` и как формируются его шаблоны?

Файл `.gitignore` используется для игнорирования файлов, не являющихся исходным кодом. Шаблоны формируются в каждой новой строке. Пример формирования шаблона: `*.class` – игнорирует все файлы с расширением `.class`, `target/` – игнорирует папку `target` и все её содержимое.

16. Что такое ветка в Git и почему её называют подвижным указателем?

Ветка — это указатель на определённый коммит. Её называют подвижным указателем, поскольку коммит содержит информацию о предыдущем коммите, при указании ветки на конкретный коммит к ней относятся и все предыдущие.

17. В чём разница между командами `git checkout master` и `git checkout -b new-branch`?

`git checkout master` – переход в ветку `master`, а `git checkout -b new-branch` – создание новой ветки `new-branch` и переход в неё.

18. Что происходит при выполнении команды `git merge` и в каком случае возникает конфликт?

При выполнении команды `git merge` происходит слияние указанной ветки в ту ветку, в которой вы сейчас находитесь. Конфликт возникает в случае, когда Git не может самостоятельно решить, какую версию кода оставить. Например, в обеих ветках была изменена одна и та же строка в одном и том же файле.

19. Как разрешить конфликт слияния и какие маркеры при этом используются в файле?

Конфликт слияния можно разрешить тремя способами:

- 1) Объединить изменения в обеих ветках;
- 2) Оставить изменения одной из веток;
- 3) Выборочное редактирование.

При этом в файле используются следующие маркеры:

- 1) <<<<<<< HEAD – под ним находятся изменения из текущей ветки;
- 2) ===== - разделитель;
- 3) >>>>>>> <имя_ветки> - над ним находятся изменения из вливаемой ветки.

20. Для чего применяется команда `git log` и что означает флаг `--oneline --graph --all`?

Команда `git log` применяется для просмотра истории коммитов в ветке.

`--oneline` - сжимает информацию о каждом коммите. Git показывает только первые 7 символов хэша и заголовок сообщения коммита.

`--graph` - рисует текстовый график. Он наглядно показывает, как ветки разделялись и где они сливались.

`--all` - заставляет Git показать историю коммитов абсолютно всех веток, а не только той, на которой вы сейчас находитесь.

Ссылка на GitHub-репозиторий: https://github.com/itsqwit/anikin_1iis41

Вывод

Я начал выполнение этой лабораторной работы с проверки наличия Git в моей системе командой `git --version`. Вывод `git version 2.53.0` дал понять, что в системе уже был предустановлен Git и можно приступать к выполнению заданий.

Командой `git config --global` я установил имя пользователя и адрес электронной почты для всех проектов. Вывод команды `git config --list` позволил убедиться в том, что все настройки были применены. После этого я создал рабочую директорию `git-practice` и перешёл в неё, затем инициализировал в этой директории локальный репозиторий командой `git init`. Здесь я создал несколько файлов и командой `git status` проверил статус репозитория, определив, что созданные файлы находятся в состоянии `untracked`. Для добавления этих файлов в репозиторий я использовал команду `git add`, что перевело их в состояние `staged`.

Я создал свой первый коммит командой `git commit -m "initial commit: added base files"`, просмотрел историю коммитов командой `git log` и зафиксировал метаданные коммита: его идентификатор, автора, время создания, указатель текущей ветки и написанный мной комментарий. После этого в ранее созданный файл `readme.txt` я внес изменения, которые отобразились в выводе команды `git diff`. Затем был создан файл `.gitignore`, в который я внёс шаблоны `*.log`, `temp/`, `*.tmp`, `.cache/` для игнорирования указанных папок и файловых форматов.

Командой `git checkout -b feature/update-readme` я создал новую ветку и перешёл в неё, в чем убедился после проверки списка веток командой `git branch -a`. В этой ветке я снова внёс изменения в файл `readme.txt`, закоммитил их, переключился на ветку `master` командой `git checkout master` и выполнил слияние новой ветки в `master` командой `git merge feature/update-readme`. Проанализировав итоговую историю командой `git log --oneline --graph --all`, я убедился, что все изменения были успешно закоммичены и сохранены в главной ветке.

Таким образом, эта лабораторная работа познакомила меня с базовыми и продвинутыми приёмами работы с распределённой системой контроля версий Git в операционной системе Linux. Я изучил процесс установки и настройки Git, освоил основные команды для работы с локальным репозиторием, научился управлять состоянием файлов, работать с ветками и выполнять слияния.